

Senior Full-Stack Engineer Take-Home Challenge

December 2025 - Home Energy Advisor

Build a simple "Home Energy Advisor" - a web application where users describe their home and receive AI-generated energy efficiency recommendations. You are allowed and encouraged to use AI coding tools for this.

Time expectation: 2-3 hours. Please scope down if needed, but document what you'd improve.

Requirements

1. Backend (Python)

Build a RESTful API with the following endpoints:

Endpoint	Description
POST /api/homes	Create a home profile (size, age, heating type, insulation, etc.)
GET /api/homes/{id}	Retrieve a home profile
POST /api/homes/{id}/advice	Generate energy-saving recommendations using an LLM

The /advice endpoint must integrate with an LLM (OpenAI, Anthropic, Local Ollama, or similar). Design your prompt to produce actionable, prioritized recommendations.

2. Frontend (Vue.js / TypeScript)

Build a simple UI that allows users to:

- Input their home details via a form
- View AI-generated recommendations
- Display results in a clear, readable format

3. Developer experience

Think about the experience of a new developer picking up this codebase. Make it easy for them, assuming you're not around to explain things.

What We're Evaluating

- API design: RESTful conventions, error handling, request/response structure
- Code quality: Readability, structure, pragmatic use of patterns
- LLM integration: Prompt design, response handling, error cases
- Frontend craft: Component structure, TypeScript usage, basic UX
- Ownership: You must understand the entire codebase even if AI helped you write it

Deliverables

1. Code repository (GitHub link or zip) with a README containing:
 - Setup instructions
 - Any assumptions or tradeoffs you made
2. AI Tool Usage Log (as a section in the README). We expect you to use AI coding assistants. Please document:
 - Which tools you used (Copilot, Claude, Cursor, etc.)
 - 2-3 examples of prompts you found effective
 - What you used AI for vs. wrote/designed yourself
 - One thing AI got wrong that you had to fix

Notes

- Use any Python framework (FastAPI, Flask, etc.) and state management approach you prefer
- Mocked LLM responses are acceptable if you don't have API access - just show how you'd structure the real integration
- Skip auth, deployment, and elaborate styling - focus on the core engineering
- A working solution with rough edges beats a polished partial solution

Good luck

We're looking forward to seeing what you come up with!